

Intro to Relational Databases

Assoc. Prof. Kitsana Waiyamai, Ph. D.
Dept. Of Computer Engineering
Faculty of Engineering, Kasetsart University
Bangkok, Thailand

Slides based on Jennifer Widom



Database Models

- Database Model:
  - "A formal definition on how to represent data and the available data operations."
- Examples of Models
  - Network model, Hierarchical model
  - Star-schema model
  - Time-Series Data Model
  - Key-Value Data Model
  - **Relational model**
  - Graph-based model
  - Semi-structured data model, for example XML
- New models:
  - No-SQL models such as MongoDB
  - Other No-SQL models

Why Relational Databases?

Imagine storing university admission data in ONE table.

Student ID	Student Name	GPA	High School Size	College	State	Enrollment	Major	Decision
123	Amy	3.9	1000	Stanford	CA	15000	CS	Y
123	Amy	3.9	1000	Stanford	CA	15000	EE	N
123	Amy	3.9	1000	Berkeley	CA	36000	CS	Y
234	Bob	3.6	1500	Berkeley	CA	36000	Biology	N

Question:
Is one big table really a good design?

Problems appear as data grows:
- Repeated student information
- Repeated college information
- Difficult maintenance

Problem 1: Data Redundancy

Student and college information are repeated many times => **Duplicated Data !**

Student ID	Student Name	GPA
123	Amy	3.9
123	Amy	3.9
123	Amy	3.9

College	State	Enrollment
Stanford	CA	15000
Stanford	CA	15000
Stanford	CA	15000

Result:

- Wasted storage
- Duplicated data

Problem 2: Update Anomaly

If Stanford enrollment changes from 15,000 ---> 17,000, multiple rows must be updated.

College	Enrollment
Stanford	15000
Stanford	15000
Stanford	15000

Missing one row causes inconsistent data.

Problem 3: Insert Anomaly

We cannot insert a new college unless at least one student applies.

The design unnecessarily couples unrelated information.

College	State	Enrollment
CMU	PA	14000

Problem 4: Delete Anomaly

If the last application to MIT is deleted => We accidentally lose MIT's information as well.

Student ID	Student Name	GPA	High School Size	College	State	Enrollment	Major	Decision
123	Amy	3.9	1000	Stanford	CA	15000	CS	Y
123	Amy	3.9	1000	Stanford	CA	15000	EE	N
123	Amy	3.9	1000	Berkeley	CA	36000	CS	Y
234	Bob	3.6	1500	Berkeley	CA	36000	Biology	N
234	Bob	3.6	1500	MIT	CA	56000	CS	N

Relational Concepts

Tables, keys, constraints, and relationships
using Student–Course–Enrollment

Student

StudentID	Name
S001	Anan
S002	Mali

Course

CourseID	Course
C101	Database
C102	Analytics

Enrollment

StudentID	CourseID
S001	C101
S002	C101

Big Picture

Relational Database = Collection of Relations

Relation

Table

Example schema

Student(StudentID, Name, Major, Year)
Course(CourseID, CourseName, Credit)
Enrollment(StudentID, CourseID, Semester, Grade)

Key idea

Data is stored in tables, and the tables are linked together using keys.

Relation = Table

Relation is a table of Data

StudentID	Name	Major	Year
S001	Anan	CPE	2
S002	Mali	SKE	1
S003	John	CPE	3

Relation name

Student

Tuple = Row

Tuple is one row of data in a table.

StudentID	Name	Major	Year
S001	Anan	CPE	2

This tuple means

Student S001 is Anan, majoring in CPE, year 2.

Attribute = Column

Attributes

StudentID
Name
Major
Year

StudentID identifies the student
Name stores student name
Major stores program
Year stores year of study

Attribute Domain = Allowed Values

Attribute	Domain
StudentID	Text => S001, S002
Name	Text
Major	CPE, SKE, AI
Year	1, 2, 3, 4
Grade	A, B+, B, C+, C, D+, D, F

Domain Violation !!!

Year = 9
→ not allowed

Attribute Domain



The set of allowed values for each attribute is called the **domain** of the attribute



Attribute values are (normally) required to be **atomic**; that is, indivisible



The special value **null** is a member of every domain. Indicated that the value is “unknown”



Later, we will see that **null** causes complications in the definition of many operations

NULL

NULL means missing, unknown, or not applicable

StudentID	CourseID	Grade
S001	C101	A
S002	C101	NULL

Meaning

S002 registered for C101, but grade is not available yet.

Important

NULL ≠ 0 • NULL ≠ blank text

NULL – special value for "unknown" or "undefined"

Student

ID	name	GPA	photo
123	Amy	3.9	
234	Bob	3.4	NULL
345	Craig	NULL	
	⋮		

College

name	state	enr
Stanford	CA	15,000
Berkeley	CA	36,000
MIT	MA	10,000
	⋮	

Schema vs Instance

Schema = Table Structure

Student(StudentID, Name, Major, Year)

Instance = Current Data

The actual data currently stored in the table.

StudentID	Name	Major	Year
S001	Anan	CPE	2
S002	Mali	SKE	1

Schema changes slowly • Instance changes frequently

Cardinality and Degree

StudentID	Name	Major	Year
S001	Anan	CPE	2
S002	Mali	SKE	1
S003	John	CPE	3

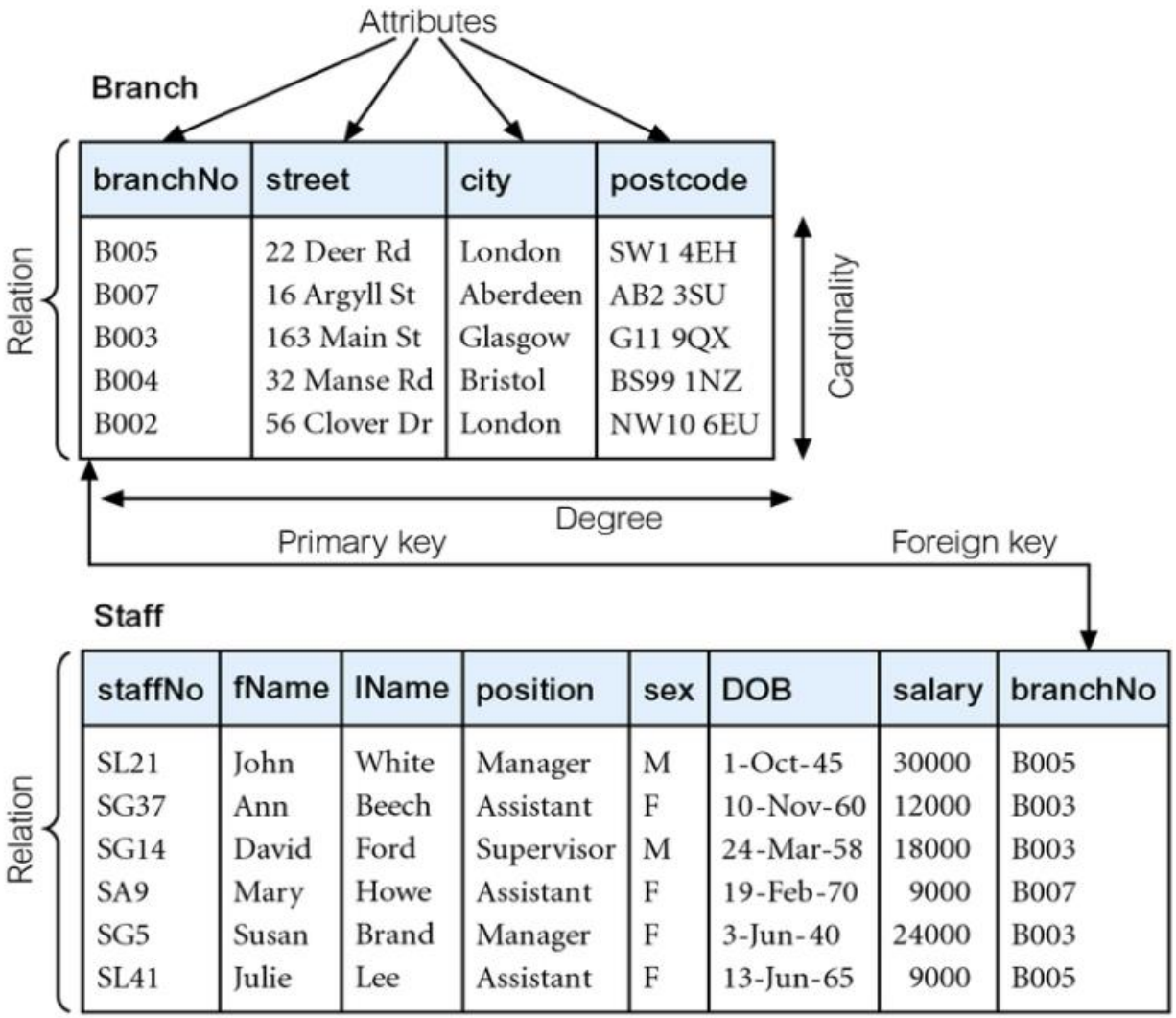
Cardinality

Number of Rows = 3

Degree

Number of Columns = 4

Example: Cardinality& Degree



Primary Key

A primary key is an attribute used to uniquely identify each row in a table.

Student schema

Student(StudentID, Name, Major, Year)

Primary Key

StudentID (PK)

StudentID	Name
S001	Anan
S002	Mali
~~S001~~	~~John~~

Problem: S001 is duplicated → not allowed.

Candidate Key = Another possible unique identifier

Student(StudentID, CitizenID, Email, Name)

Possible candidate keys

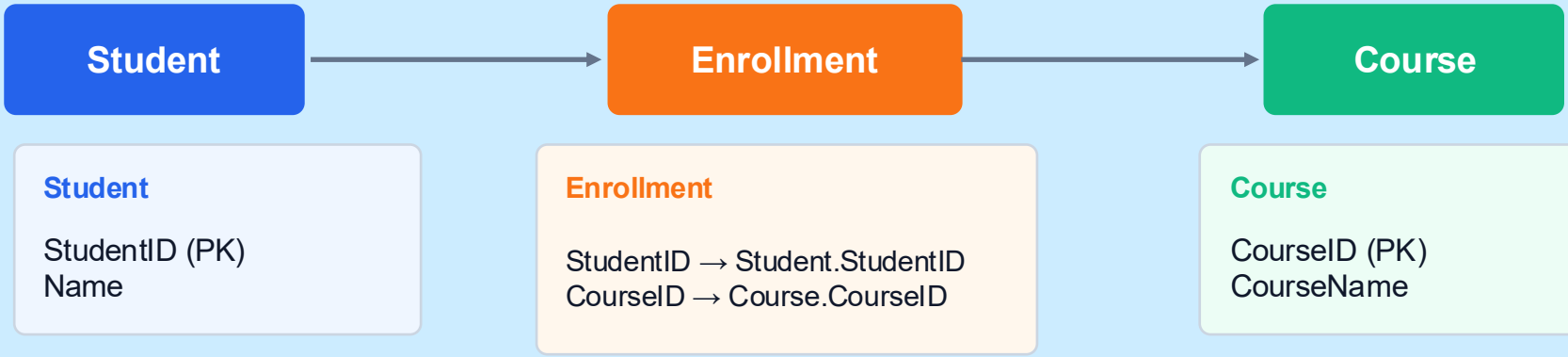
StudentID
CitizenID
Email

Choose one as Primary Key

Primary Key = StudentID

Many possible unique identifiers

Foreign Key is an attribute that references the primary key of another table.



StudentID	Name
S001	Anan
S002	Mali

CourseID	CourseName
C101	Database
C102	Data Analytics

StudentID	CourseID	Grade
S001	C101	A
S001	C102	B+
S002	C101	B

Enrollment tells us: Which student takes which course.

Why Foreign Key Matters

StudentID	CourseID	Grade
S999	C101	A

Problem

S999 does not exist in Student table

Can a student enroll in a course if the student does not exist?

Integrity Constraints

Integrity Constraints are rules that keep data correct.

Entity Integrity

Primary Key cannot be NULL.

StudentID cannot be NULL

Referential Integrity

Foreign Key must refer to existing data.

In Enrollment table, StudentID must exist in Student.StudentID

Domain Constraint

Values must be valid.

Grade must be A, B+, B, C+, C, D+, D, F

Start Thinking About Relationships

We have 3 tables:

Student

Enrollment

Course

Question

What is the relationship between Student and Course?
1-to-1? 1-to-Many? Many-to-Many?

Student and Course

Ask students

Can one student take many courses?

Answer

Yes

Example

S001 → Database
S001 → Data Analytics
S001 → Programming

One Student → Many Courses

Course and Student

Ask students

Can one course have many students?

Answer

Yes

Example

Database → S001
Database → S002
Database → S003

One Course → Many Students

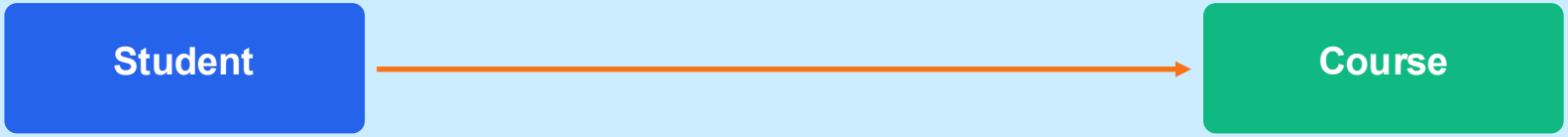
Student–Course Relationship

From Student side

One student can take many courses.

From Course side

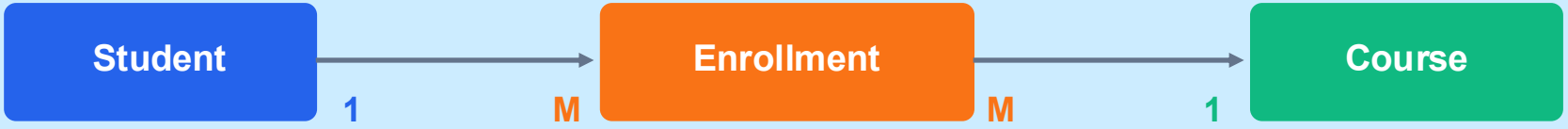
One course can have many students.



Many-to-Many (M:N)

Why We Need Enrollment

Relational database cannot directly store M:N relationship in one clean table.



Enrollment converts

Student ↔ Course (Many-to-Many) => Student → Enrollment and Course → Enrollment

Enrollment is needed because Student and Course have a Many-to-Many relationship.

Enrollment as Relationship Table

StudentID	CourseID	Semester	Grade
S001	C101	1/2026	A
S001	C102	1/2026	B+
S002	C101	1/2026	B

Each row means

One student enrolls in one course in one semester. And the **relationship itself can have attributes**

Key Idea

Student

stores student information

Course

stores course information

Enrollment

stores the relationship between them

Without Enrollment

We cannot properly represent many students taking many courses.

With Enrollment

Data becomes clean, flexible, and consistent.

Summary

Relational Concepts

Relation = Table
Tuple = Row
Attribute = Column
Attribute Domain = Allowed values
Schema = Table structure
Instance = Current data
Cardinality = Number of rows
Degree = Number of columns

Keys & Rules

Primary Key = Unique identifier
Candidate Key = Possible primary key
Foreign Key = Link between tables
NULL = Unknown / missing value
Integrity Constraints = Rules for correct data

Relational Database Model

Database = set of named **relations** (or **tables**)
Each relation has a set of **attributes** (or **columns**)

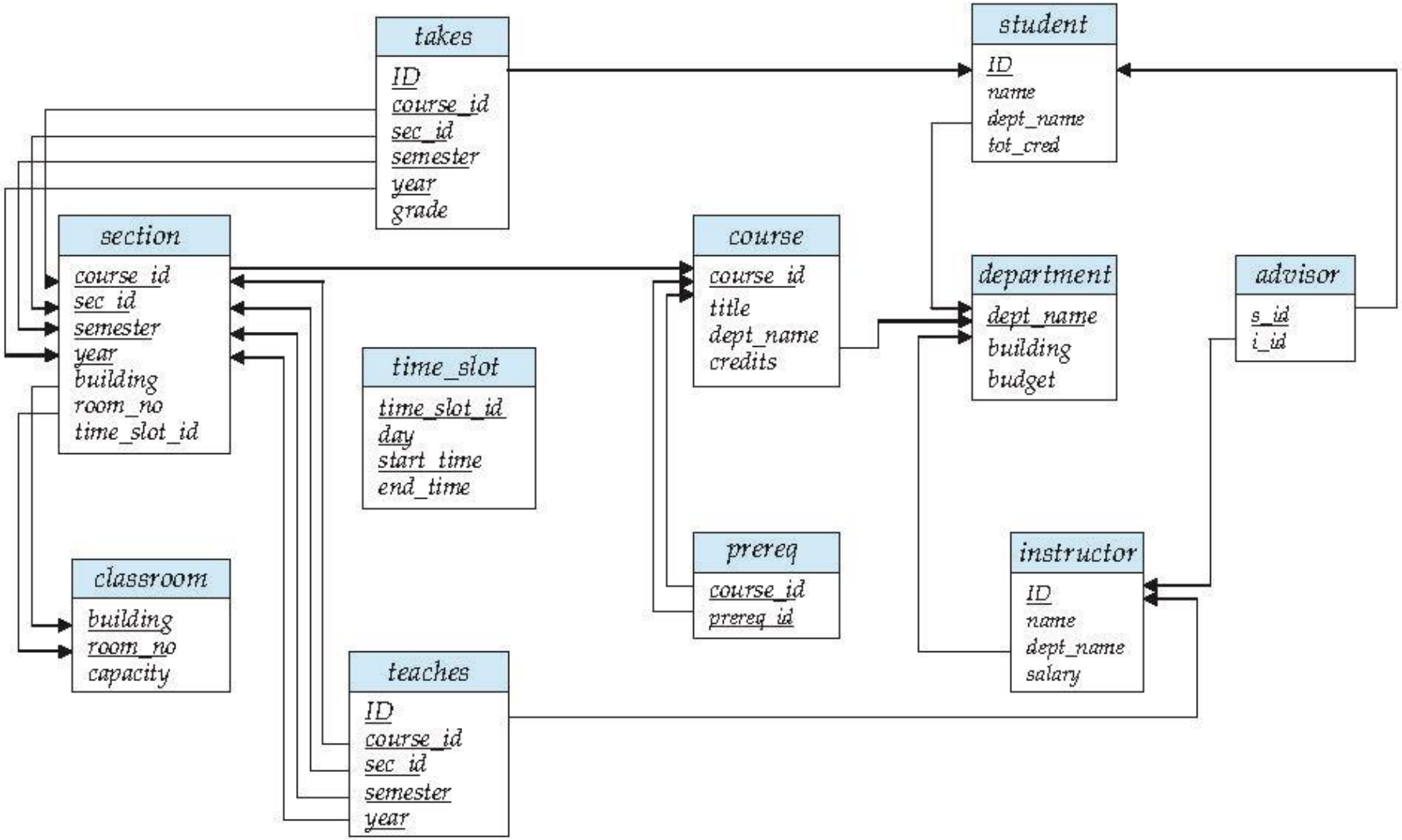
Relational Schema = Structural description of relations in database

Logical Schema of the University Database

classroom(<u>*building*</u>, <u>*room_number*</u>, *capacity*)
department(<u>*dept_name*</u>, *building*, *budget*)
course(<u>*course_id*</u>, *title*, *dept_name*, *credits*)
instructor(<u>*ID*</u>, *name*, *dept_name*, *salary*)
section(<u>*course_id*</u>, <u>*sec_id*</u>, <u>*semester*</u>, <u>*year*</u>, *building*, *room_number*, *time_slot_id*)
teaches(<u>*ID*</u>, <u>*course_id*</u>, <u>*sec_id*</u>, <u>*semester*</u>, <u>*year*</u>)
student(<u>*ID*</u>, *name*, *dept_name*, *tot_cred*)
takes(<u>*ID*</u>, <u>*course_id*</u>, <u>*sec_id*</u>, <u>*semester*</u>, <u>*year*</u>, *grade*)
advisor(<u>*s_ID*</u>, *i_ID*)
time_slot(<u>*time_slot_id*</u>, <u>*day*</u>, <u>*start_time*</u>, *end_time*)
prereq(<u>*course_id*</u>, <u>*prereq_id*</u>)

do this format in homework

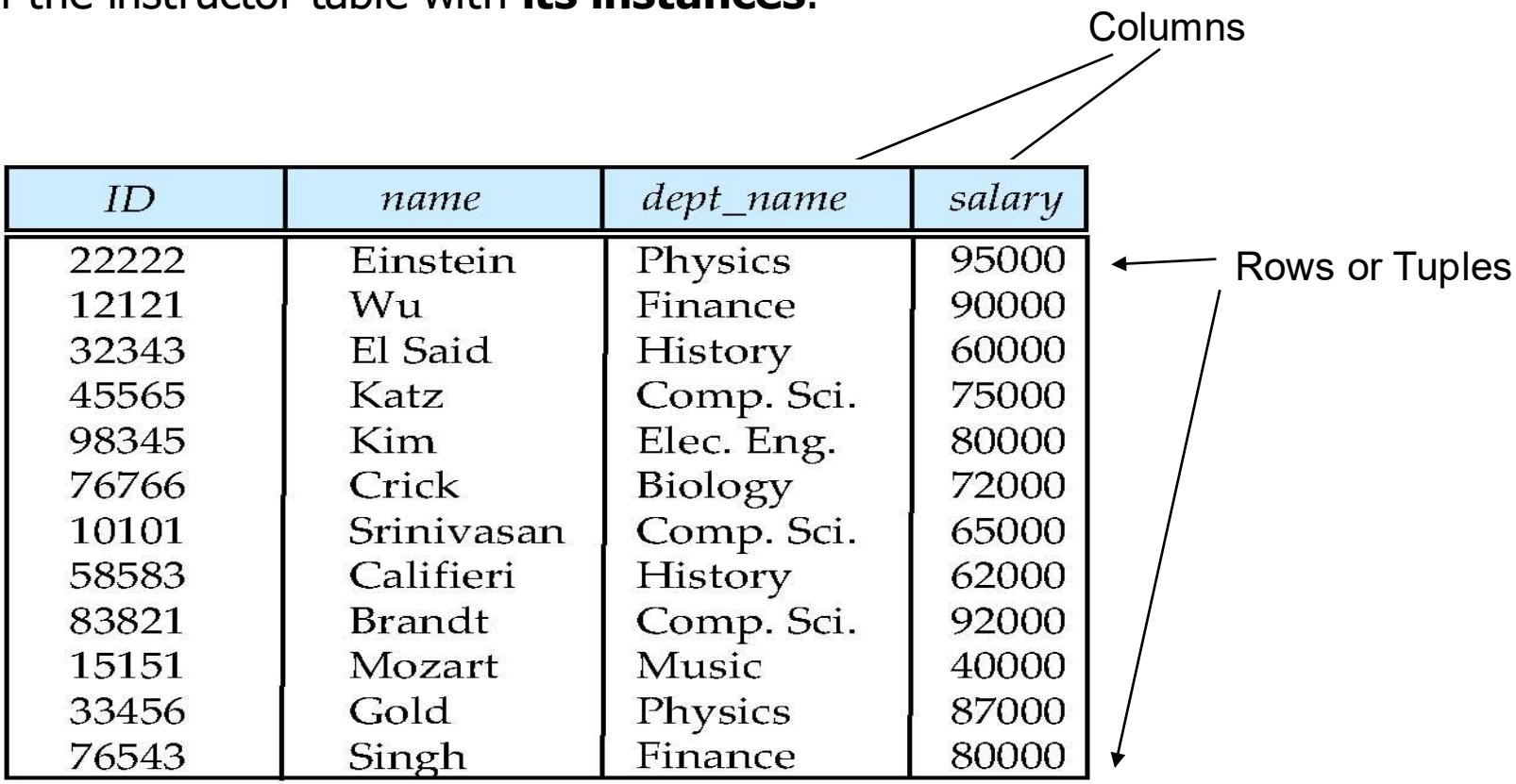
Relational Schema of the University Database



Relational Databases

Data stored in various tables.

Example of the instructor table with **its instances**.



ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

Each **tuple** (or **row**) has a value for each attribute
Each **column** (or attribute) has a **type** (or **domain**)

Relational Databases

- Relation (table)
- Attribute (column)
- Tuple (row)
- Types and domains

Cities

city	country	latitude	longitude	temperature
Aalborg	Denmark	57.03	9.92	7.52
Aberdeen	United Kingdom	57.17	-2.08	8.1
Abisko	Sweden	63.35	18.83	0.2
Adana	Turkey	36.99	35.32	18.67
Albacete	Spain	39.0	-1.87	12.62
Algeciras	Spain	36.13	-5.47	17.38
Amiens	France	49.9	2.3	10.17
Amsterdam	Netherlands	52.35	4.92	8.93
Ancona	Italy	43.6	13.5	13.52
Andorra	Andorra	42.5	1.52	9.6
Angers	France	47.48	-0.53	10.98
Ankara	Turkey	39.93	32.86	9.86
Antalya	Turkey	36.89	30.7	11.88
Arad	Romania	46.17	21.32	9.32
Athens	Greece	37.98	23.73	17.41
Augsburg	Germany	48.35	10.9	4.54

Relational Databases

- Used by all major commercial database systems
- Very simple model
- Query with high-level languages: Highly expressive
- Efficient implementations

classroom(<u>*building*</u>, <u>*room_number*</u>, *capacity*)
department(<u>*dept_name*</u>, *building*, *budget*)
course(<u>*course_id*</u>, *title*, *dept_name*, *credits*)
instructor(<u>*ID*</u>, *name*, *dept_name*, *salary*)
section(<u>*course_id*</u>, <u>*sec_id*</u>, <u>*semester*</u>, <u>*year*</u>, *building*, *room_number*, *time_slot_id*)
teaches(<u>*ID*</u>, <u>*course_id*</u>, <u>*sec_id*</u>, <u>*semester*</u>, <u>*year*</u>)
student(<u>*ID*</u>, *name*, *dept_name*, *tot_cred*)
takes(<u>*ID*</u>, <u>*course_id*</u>, <u>*sec_id*</u>, <u>*semester*</u>, <u>*year*</u>, *grade*)
advisor(<u>*s_ID*</u>, *i_ID*)
time_slot(<u>*time_slot_id*</u>, <u>*day*</u>, <u>*start_time*</u>, *end_time*)
prereq(<u>*course_id*</u>, <u>*prereq_id*</u>)

University Database Schema

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Instances of the Instructor relation

Relations and Their Instances

Database Instances
=
The actual data currently stored in the table

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

dept_name	*building*	*budget*
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table

College Admission Database

College(cName,state,enrollment)

Student(sID,sName,GPA,sizeHS)

Apply(sID,cName,major,decision)

	cName	state	enrollment
1	Stanford	CA	15000
2	Berkeley	CA	36000
3	MIT	MA	10000
4	Cornell	NY	21000

	sID	sName	GPA	sizeHS
1	123	Amy	3.9	1000
2	234	Bob	3.6	1500
3	345	Craig	3.5	500
4	456	Doris	3.9	1000
5	567	Edward	2.9	2000
6	678	Fay	3.8	200
7	789	Gary	3.4	800
8	987	Helen	3.7	800
9	876	Irene	3.9	400
10	765	Jay	2.9	1500
11	654	Amy	3.9	1000
12	543	Craig	3.4	2000

	sID	cName	major	decision
1	123	Stanford	CS	Y
2	123	Stanford	EE	N
3	123	Berkeley	CS	Y
4	123	Cornell	EE	Y
5	234	Berkeley	biology	N
6	345	MIT	bioengineering	Y
7	345	Cornell	bioengineering	N
8	345	Cornell	CS	Y
9	345	Cornell	EE	N
10	678	Stanford	history	Y
11	987	Stanford	CS	Y
12	987	Berkeley	CS	Y
13	876	Stanford	CS	N
14	876	MIT	biology	Y
15	876	MIT	marine biology	N
16	765	Stanford	history	Y
17	765	Cornell	history	N
18	765	Cornell	psychology	Y
19	543	MIT	CS	N

Key (or Super-Key)

- **Key (or Super-key)**
  - Attribute whose value is **unique in each row** or
  - Set of attributes (**composite-key**) whose combined values are unique
- **Example of Instructor**
  - **{ID} is** superkey
    - No two instructors share the same ID
    - **{ID} is** UNIQUE
  - **{ID}** and **{ID,name}** are both superkeys of instructor
    - Since {ID} alone can uniquely identify each row, adding more attributes still maintains uniqueness =>
    - **Any superset of a superkey is also a superkey**
  - **{name} is not** superkey
    - name alone cannot uniquely identify a single instructor
  - **{name,dept_name}** is another possible superkey of instructors => Even though names repeat, the name+department combination is unique

Instructor Table

ID	name	dept_name	salary
101	Smith	Computer Science	75000
102	Jones	Mathematics	68000
103	Smith	Physics	82000
104	Brown	Computer Science	71000
105	Davis	Mathematics	69000
106	Brown	Physics	77000
107	Garcia	Computer Science	73000

Key (or Super-Key)

- **Super-Keys** of Student Table**:**

=> {sID}, {sID, Sname}

  - {**sID**} is a superkey because no two students share the same sID.
  - Another superkey is {**sID**, **sName**}.
- Even though sName alone may not uniquely identify a student, combining it with sID still results in a unique identification.
- A superkey can include extra columns that are not necessary for uniqueness.

- **College Table:**

cName	state
MIT	MA
Stanford	CA
Harvard	MA

- **Student Table:**

sID	sName
1	John Smith
2	Jane Doe
3	Alice Brown

- **Apply Table:**

sID	cName	major
1	MIT	CS
2	Stanford	Biology
3	Harvard	Economics

Candidate Keys

- **Possible superkeys** of Instructor are {ID},{ID,name},{ID,name,dept_name},{name,dept_name}
- Superkey *K* is a **candidate key** if *K* is minimal
- {*ID*} is a candidate key for *Instructor*
- {name, dept_name} is **another candidate key** for *Instructor*
- {ID, name} is not **candidate key** , since the attribute ID alone is a candidate key

Instructor Table

ID	name	dept_name	salary
101	Smith	Computer Science	75000
102	Jones	Mathematics	68000
103	Smith	Physics	82000
104	Brown	Computer Science	71000
105	Davis	Mathematics	69000
106	Brown	Physics	77000
107	Garcia	Computer Science	73000

- A **candidate key is a minimal superkey**, meaning it's a superkey with no unnecessary columns.
- Every candidate key is a superkey, but not all superkeys are candidate keys.

Primary Keys

- **A primary key** is a candidate key that is chosen by the database designer to uniquely identify rows in a table. It ensures that no two rows can have the same primary key value, and it cannot contain NULL values.
- Both {ID} and {name, dept_name} are **candidate keys**.
- One of the candidate keys is selected to be the **primary key**
- Which one?
- **{ID} can be selected as the primary key** for the following reasons:
  - **Simplicity**: Single attribute vs. Composite key - easier to reference and manage
  - **Stability**: ID numbers typically don't change over time, while names might change (marriage, legal name changes) and department assignments can change
  - **Performance**: Single-column indexes are generally faster than multi-column indexes for joins and lookups
  - **Foreign Key References**: Other tables that need to reference instructors (like a "courses" table) can use a simple single-column foreign key rather than a composite foreign key
  - **Data Entry**: Less prone to data entry errors - typing one ID number vs. ensuring both name and department are entered correctly
  - **Storage Efficiency**: Integer IDs typically take less storage space than text combinations
  - Other reasons …

Example: College Admission Database

College(cName,state,enrollment) => College name is unique

Student(sID,sName,GPA,sizeHS) => sID is unique

Apply(sID,cName,major,decision) => students only apply to each college for a particular major **one time** (and not more!)

- College Table:

cName	state	enrollment
MIT	MA	5000
Stanford	CA	7000
Harvard	MA	6000

- Student Table:

sID	sName	GPA	sizeHS
1	John Smith	3.8	1000
2	Jane Doe	3.9	800
3	Alice Brown	3.7	1200

Super-keys:

- {sID, cName, major}
- {sID, cName, major, decision}

Candidate-key:

- {sID, cName, major}

sID	cName	major	decision
1	MIT	CS	Accepted
1	MIT	Mathematics	Rejected
2	Stanford	Biology	Rejected
3	Harvard	Economics	Accepted
3	MIT	CS	Accepted

Example: College Admission Database

College(cName,state,enrollment) => College name is unique

Student(sID,sName,GPA,sizeHS) => sID is unique

Apply(sID,cName,major,decision) => students only apply to each college for a particular major **one time** (and not more!)

- College Table:

cName	state	enrollment
MIT	MA	5000
Stanford	CA	7000
Harvard	MA	6000

- Student Table:

sID	sName	GPA	sizeHS
1	John Smith	3.8	1000
2	Jane Doe	3.9	800
3	Alice Brown	3.7	1200

Super-keys:

- {sID, cName, major}
- {sID, cName, major, decision}

Candidate-key:

- {sID, cName, major}

sID	cName	major	decision
1	MIT	CS	Accepted
1	MIT	Mathematics	Rejected
2	Stanford	Biology	Rejected
3	Harvard	Economics	Accepted
3	MIT	CS	Accepted

Exercise-0.1

Foreign Keys

- **Foreign key**
  - is a column (or a set of columns) in one table that refers to the **primary key in another table**.
- **Foreign key**: an attribute in a table used to define its relationship with another table
- **Foreign key constraint**: Value in one relation **must appea**r in another relation !
- Example, in Instructor
  - Instructor {**ID**} is a **primary key**
  - **dept_name** is **a foreign key** from Instructor referencing Department
    - Instructor is a **Referencing** relation
    - Department is a **Referenced** relation
  - **dept_name** in Instructor must exist in Department relation
  - **Cannot insert instructor** with dept_name = "Art History" (**Non existing Department**)

ID	name	dept_name	salary
101	Smith	Computer Science	75000
102	Jones	Mathematics	68000
103	Wilson	Computer Science	82000
104	Brown	Physics	71000
105	Davis	Mathematics	69000

Instructor(ID,name,dept_name,salary)

dept_name	building	budget
Computer Science	Taylor	100000
Mathematics	Watson	80000
Physics	Watson	70000
Biology	Painter	90000

Department (dept_name,building,budget)

Exercise-0.2

Foreign Keys

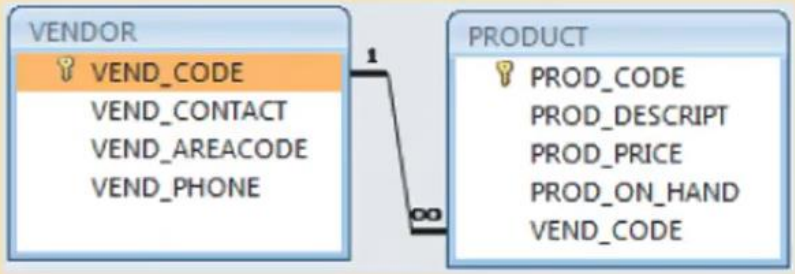
Table name: PRODUCT **Database name:**
Primary key: PROD_CODE
Foreign key: VEND_CODE

PROD_CODE	PROD_DESCRIPT	PROD_PRICE	PROD_ON_HAND	VEND_CODE
001278-AB	Claw hammer	12.95	23	232
123-21UUY	Houselite chain saw, 16-in. bar	189.99	4	235
QER-34256	Sledge hammer, 16-lb. head	18.63	6	231
SRE-657UG	Rat-tail file	2.99	15	232
ZZX/3245Q	Steel tape, 12-ft. length	6.79	8	235

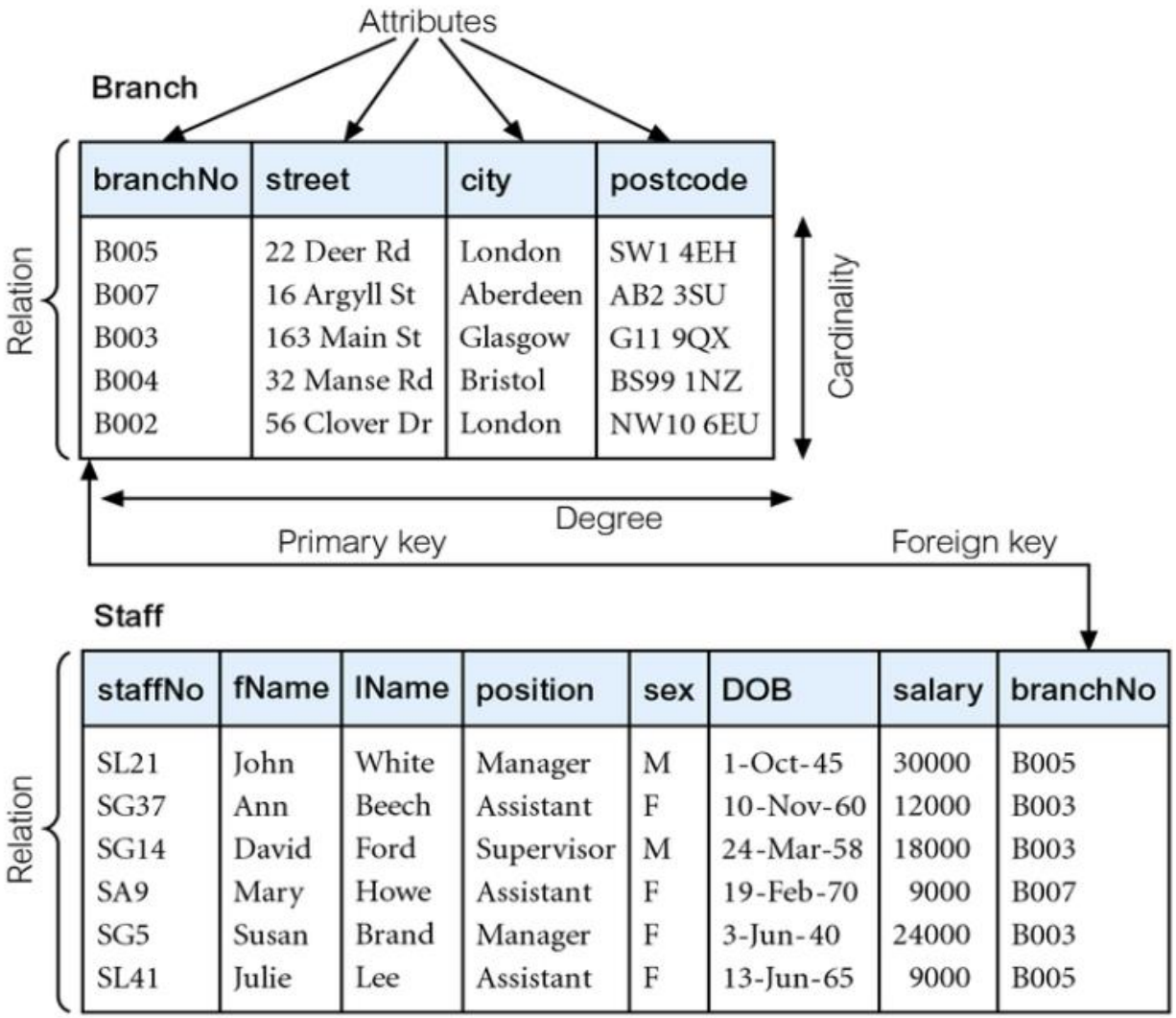
link

Table name: VENDOR
Primary key: VEND_CODE
Foreign key: none

VEND_CODE	VEND_CONTACT	VEND_AREACODE	VEND_PHONE
230	Shelly K. Smithson	608	555-1234
231	James Johnson	615	123-4536
232	Annelise Crystall	608	224-2134
233	Candice Wallace	904	342-6567
234	Arthur Jones	615	123-3324
235	Henry Ortozo	615	899-3425



Example: Primary Keys & Foreign Keys



Relational Query Languages

- **Pure** or **Formal** Languages:
  - **Relational algebra**
  - Tuple relational calculus
  - Domain relational calculus
- They are pure languages that are equivalent in computing power
- **High-Level Querying** Languages:
  - **Relational algebra => SQL**
  - **SQL** is implemented as high-level query language
  - **SQL** is most widely used commercial language
  - Application programs generally access databases through
    - **Language extensions** to allow embedded SQL
    - **Application program interface** (e.g., ODBC/JDBC) which allow SQL queries to be sent to a database

Relational Algebra vs SQL

- Relational Algebra — formal

$\Pi_{ID}\, \sigma_{GPA>3.7 \wedge cName='Stanford'}(Student \bowtie Apply)$

- SQL — actual / implemented

```
Select Student.ID
From Student, Apply
Where Student.ID=Apply.ID
And GPA>3.7 and college='Stanford'
```

IDs of students with GPA > 3.7 applying to Stanford

Relational Algebra

Query (expression) on set of relations produces relation as a result

- Simplest query => **relation name**

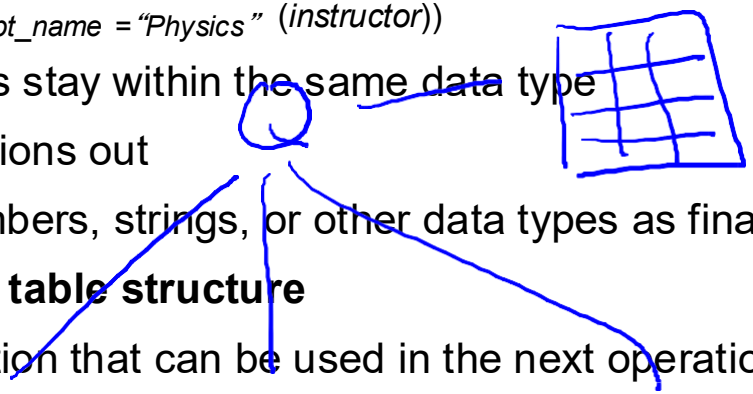
Instructor

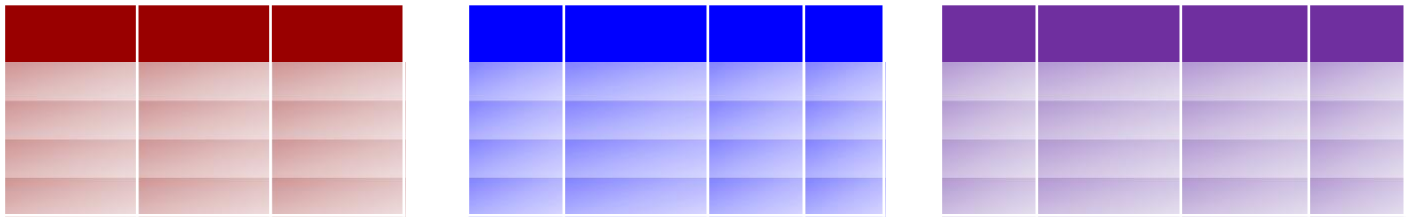
ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

- Use operators to filter, slice, combine
- **Operators**: select, project, cross-product, theta-join, natural-join, …

Relational Algebra Properties

- **Properties : Composition + Closed**
- Query (or expression) on set of relations produces → relation
  - **Input**: One or More relations
  - **Output**: Always a Single Relation
- **Composition**: You can chain operations together
  - **Output of one query becomes input to another query**
  - **Example**: $\Pi_{name}(\sigma_{dept_name = \text{“Physics”}}(instructor))$
- **Closed**: Operations always stay within the same data type
  - Relations in → Relations out
  - Never produces numbers, strings, or other data types as final result
  - **Always produces a table structure**
- Each step produces a relation that can be used in the next operation.





Relational Algebra based on Sets and Un-ordered

- Sets with no duplicates
- Rows are in an arbitrary order
- **Example**: *instructor* relation with unordered rows (or instances)

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

- Relational Algebra
  - $\Rightarrow$ **Sets**
  - $\Rightarrow$ **No duplicates**
- SQL
  - $\Rightarrow$ **Multi-Sets, Bags**
  - $\Rightarrow$ **With Duplicates**

Relational Algebra Operators

- A procedural language consisting of a set of operations that take relations as input and produce a new relation as their result.
- Basic operators: Unary & Binary operators
  - Selection: σ
  - Projection: π
  - Rename: ρ
  - Union: ∪
  - Intersection: ∩
  - Difference: −
  - Cartesian Product: ×
- Other operators
  - Join: ⋈
  - Division: ÷
  - Etc.

Operator	Symbol	Unicode
Selection	σ	U+03C3
Projection	π	U+03C0
Rename	ρ	U+03C1
Union	∪	U+222A
Intersection	∩	U+2229
Difference	−	U+2212
Cartesian Product	×	U+00D7
Join	⋈	U+22C8
Division	÷	U+00F7

Selection

- The **selection** operation **selects rows that satisfy a given predicate**.
- Notation: $\sigma_p(r)$
- p is called the **selection predicate**

- **Example**: select those rows of the *instructor* relation where the instructor is in the "Physics" department.
  - Query

$$\sigma_{dept_name = \text{“Physics”}}(instructor)$$

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

Selection

- The **selection** operation **selects rows that satisfy a given predicate**.
- Notation: $\sigma_p(r)$
- p is called the **selection predicate**

- **Example**: select those rows of the *instructor* relation where the instructor is in the "Physics" department.
  - Query

$$\sigma_{dept_name = \text{“Physics”}}(instructor)$$

- Result

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

```
SELECT *
FROM instructor
WHERE dept_name = "Physics"
```

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

Selection

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000.00	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000.00	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000.00	B003
SA9	Mary	Howe	Assistant	F	19-Feb-70	9000.00	B007
SG5	Susan	Brand	Manager	F	3-Jun-40	24000.00	B003
SL41	Julie	Lee	Assistant	F	13-Jun-65	9000.00	B005

$\sigma_{salary > 10000}$ (Staff)

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24- Mar-58	18000	B003
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003

Result => Relation with same number of columns

Selection

- Comparisons using =, ≠, >, ≥, <, ≤ in the selection predicate.
- Combine several predicates into a larger predicate by using the connectives:

  ∧ (**and**), ∨ (**or**), ¬ (**not**)
- **Example**: Find the instructors in Physics with a salary greater $90,000, we write:

$$\sigma_{dept_name = \text{“Physics”} \wedge salary > 90{,}000}(instructor)$$

dept_name	*building*	*budget*
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table

Example: College Admission Database

College(cName,state,enrollment)

Student(sID,sName,GPA,sizeHS)

Apply(sID,cName,major,decision)

	cName	state	enrollment
1	Stanford	CA	15000
2	Berkeley	CA	36000
3	MIT	MA	10000
4	Cornell	NY	21000

	sID	sName	GPA	sizeHS
1	123	Amy	3.9	1000
2	234	Bob	3.6	1500
3	345	Craig	3.5	500
4	456	Doris	3.9	1000
5	567	Edward	2.9	2000
6	678	Fay	3.8	200
7	789	Gary	3.4	800
8	987	Helen	3.7	800
9	876	Irene	3.9	400
10	765	Jay	2.9	1500
11	654	Amy	3.9	1000
12	543	Craig	3.4	2000

	sID	cName	major	decision
1	123	Stanford	CS	Y
2	123	Stanford	EE	N
3	123	Berkeley	CS	Y
4	123	Cornell	EE	Y
5	234	Berkeley	biology	N
6	345	MIT	bioengineering	Y
7	345	Cornell	bioengineering	N
8	345	Cornell	CS	Y
9	345	Cornell	EE	N
10	678	Stanford	history	Y
11	987	Stanford	CS	Y
12	987	Berkeley	CS	Y
13	876	Stanford	CS	N
14	876	MIT	biology	Y
15	876	MIT	marine biology	N
16	765	Stanford	history	Y
17	765	Cornell	history	N
18	765	Cornell	psychology	Y
19	543	MIT	CS	N

Example: College Admission Database

College(**cName**,**state**,**enrollment**) => College name is unique

Student(**sID**,**sName**,**GPA**,**sizeHS**) => SID is unique

Apply(**sID**,**cName**,**major**,**decision**) => students will only apply to each college for a particular major one time

College

cName	state	enr

Student

sID	sName	GPA	HS

Apply

sID	cName	major	dec

Exercise-1

Projection

- A unary operation that returns its argument relation, with **certain attributes left out**.
- **Notation**:

$$\Pi_{A_1,A_2,A_3 \ldots A_k} (r)$$

where A_1, A_2 are attribute names and r is a relation name.

- The result is defined as the relation of k columns obtained by erasing the columns that are not listed

- **Since relations are sets => Duplicate rows removed** from result

Projection

- **Example**: eliminate the *dept_name* attribute of *instructor*
- **Query**:

$$\Pi_{ID,\ name,\ salary}\ (instructor)$$

- **Result** contains same number of rows of the input relation *instructor*

ID	*name*	*salary*
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000
22222	Einstein	95000
32343	El Said	60000
33456	Gold	87000
45565	Katz	75000
58583	Califieri	62000
76543	Singh	80000
76766	Crick	72000
83821	Brandt	92000
98345	Kim	80000

Composition of Operations

- **Result** of a relational-algebra operation is always a **relation**
- Relational-algebra operations can be **composed** together into a **relational-algebra expression**.
- Instead of giving the name of a relation as the argument of the **projection** operation, we give **an expression that evaluates to a relation**.

- Example: **Selection** with **Projection**
- Find the names of all instructors in the Physics department.

$$\Pi_{name}(\sigma_{dept_name = \text{“Physics”}}(instructor))$$

Exercise-2

Combining relations

- Cartesian-product (or Cross-product): $\times$
- Natural Join: $\bowtie$
- Theta-Join: $\bowtie_{cond}$

Cartesian-Product (or Cross-Product)

- The Cartesian-product (×) allows us to combine information from any two relations R1 & R2.
- Concatenate rows of R1 with rows of R2
- Number of resulting rows = number of rows(R1) X number of rows(R2)

SubjectId
com101
com102
com211

SUBJECT

StudentId
s001
s002

STUDENT

SubjectId	StudentId
com101	s001
com101	s002
com102	s001
com102	s002
com211	s001
com211	s002

SUBJECT x STUDENT

Cartesian-Product

- **Example**: Cartesian product of *instructor* and t*eaches* is written as:

  instructor X *teaches*

- We construct a tuple of the result out of **each possible pair of tuples**: one from the *instructor* relation and one from the *teaches* relation
  - **Associates every tuple of instructor with every tuple of teaches**
- Since the instructor *ID* appears in both relations **we distinguish between these attribute by attaching to the attribute the name of the relation from which the attribute originally came**.

classroom(<u>*building*</u>, <u>*room_number*</u>, *capacity*)
department(<u>*dept_name*</u>, *building*, *budget*)
course(<u>*course_id*</u>, *title*, *dept_name*, *credits*)
instructor(<u>*ID*</u>, *name*, *dept_name*, *salary*)
section(<u>*course_id*</u>, <u>*sec_id*</u>, <u>*semester*</u>, <u>*year*</u>, *building*, *room_number*, *time_slot_id*)
teaches(<u>*ID*</u>, <u>*course_id*</u>, <u>*sec_id*</u>, <u>*semester*</u>, <u>*year*</u>)
student(<u>*ID*</u>, *name*, *dept_name*, *tot_cred*)
takes(<u>*ID*</u>, <u>*course_id*</u>, <u>*sec_id*</u>, <u>*semester*</u>, <u>*year*</u>, *grade*)
advisor(<u>*s_ID*</u>, <u>*i_ID*</u>)
time_slot(<u>*time_slot_id*</u>, <u>*day*</u>, <u>*start_time*</u>, *end_time*)
prereq(<u>*course_id*</u>, <u>*prereq_id*</u>)

Instructor.ID	*name*	*dept_name*	*salary*	*teaches.ID*	*course_id*	*sec_id*	*semester*	*year*

The *instructor* x *teaches* table

Instructor.ID	*name*	*dept_name*	*salary*	*teaches.ID*	*course_id*	*sec_id*	*semester*	*year*
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	12121	FIN-201	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	15151	MU-199	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	22222	PHY-101	1	Fall	2017
…	…	…	…	…	…	…	…	…
…	…	…	…	…	…	…	…	…
12121	Wu	Finance	90000	10101	CS-101	1	Fall	2017
12121	Wu	Finance	90000	10101	CS-315	1	Spring	2018
12121	Wu	Finance	90000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
12121	Wu	Finance	90000	15151	MU-199	1	Spring	2018
12121	Wu	Finance	90000	22222	PHY-101	1	Fall	2017
…	…	…	…	…	…	…	…	…
…	…	…	…	…	…	…	…	…
15151	Mozart	Music	40000	10101	CS-101	1	Fall	2017
15151	Mozart	Music	40000	10101	CS-315	1	Spring	2018
15151	Mozart	Music	40000	10101	CS-347	1	Fall	2017
15151	Mozart	Music	40000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
15151	Mozart	Music	40000	22222	PHY-101	1	Fall	2017
…	…	…	…	…	…	…	…	…
…	…	…	…	…	…	…	…	…
22222	Einstein	Physics	95000	10101	CS-101	1	Fall	2017
22222	Einstein	Physics	95000	10101	CS-315	1	Spring	2018
22222	Einstein	Physics	95000	10101	CS-347	1	Fall	2017
22222	Einstein	Physics	95000	12121	FIN-201	1	Spring	2018
22222	Einstein	Physics	95000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
…	…	…	…	…	…	…	…	…
…	…	…	…	…	…	…	…	…

classroom(*building*, *room_number*, *capacity*)
department(*dept_name*, *building*, *budget*)
course(*course_id*, *title*, *dept_name*, *credits*)
instructor(*ID*, *name*, *dept_name*, *salary*)
section(*course_id*, *sec_id*, *semester*, *year*, *building*, *room_number*, *time_slot_id*)
teaches(*ID*, *course_id*, *sec_id*, *semester*, *year*)
student(*ID*, *name*, *dept_name*, *tot_cred*)
takes(*ID*, *course_id*, *sec_id*, *semester*, *year*, *grade*)
advisor(*s_ID*, *i_ID*)
time_slot(*time_slot_id*, *day*, *start_time*, *end_time*)
prereq(*course_id*, *prereq_id*)

Cartesian-Product with condition

- The Cartesian-Product

  instructor X *teaches*

  associates every tuple of instructor with every tuple of teaches.

  - Most of the resulting rows have information about instructors who did NOT teach a particular course.

- To get only those tuples of "*instructor* X *teaches* " that relate to instructors and the courses that they taught, we write:

  $$\sigma_{instructor.id = teaches.id} (instructor \times teaches))$$

  - We get only those tuples of "*instructor* X *teaches*" that relate to **instructors and the courses that they taught**.

- The result of this expression, shown in the next slide

Cartesian-Product with condition

- The table corresponding to:

$$\sigma_{instructor.id = teaches.id}(instructor \times teaches))$$

classroom(building, room_number, capacity)
department(dept_name, building, budget)
course(course_id, title, dept_name, credits)
instructor(ID, name, dept_name, salary)
section(course_id, sec_id, semester, year, building
teaches(ID, course_id, sec_id, semester, year)
student(ID, name, dept_name, tot_cred)
takes(ID, course_id, sec_id, semester, year, grade)
advisor(s_ID, i_ID)
time_slot(time_slot_id, day, start_time, end_time)
prereq(course_id, prereq_id)

Instructor.ID	name	dept_name	salary	teaches.ID	course_id	sec_id	semester	year
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
32343	El Said	History	60000	32343	HIS-351	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-101	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-319	1	Spring	2018
76766	Crick	Biology	72000	76766	BIO-101	1	Summer	2017
76766	Crick	Biology	72000	76766	BIO-301	1	Summer	2018
83821	Brandt	Comp. Sci.	92000	83821	CS-190	1	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-190	2	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-319	2	Spring	2018
98345	Kim	Elec. Eng.	80000	98345	EE-181	1	Spring	2017

Exercise-3

Natural Join Operation

Natural-Join: ⋈

- **Automatic condition**: Joins on all attributes with the same name
- **No explicit condition needed** => **Implicit equality**
- **Eliminate duplicate attributes**

Natural Join ⋈

- Enforce equality on all attributes with same name
  - Combine attributes where the same attribute names
- Eliminate duplicate attributes with same names

⋈ (Natural Join)	*instructor* ⋈ *department*
	Output pairs of rows from the two input relations that have the same value on all attributes that have the same name.

Natural-Join: ⋈

Natural Join ⋈

$$Student \bowtie Apply$$

- Enforce equality on all attributes with same name
  - Combine attributes where the same attribute names
- Eliminate duplicate attributes with same names

Duplicate attributes appear only once (no duplicate column)

- We get a new table that combines columns from both tables where sID matches.
- The result combined each row in the Student table with the corresponding rows in the Apply table where the sID values match.

Student Table:

sID	sName	GPA	sizeHS
1	John Smith	3.8	1000
2	Jane Doe	3.9	800
3	Alice Brown	3.7	1200

Apply Table:

sID	cName	major	decision
1	MIT	CS	Accepted
2	Stanford	Biology	Rejected
3	Harvard	Economics	Accepted
1	UCLA	Physics	Accepted

sID	sName	GPA	sizeHS	cName	major	decision
1	John Smith	3.8	1000	MIT	CS	Accepted
1	John Smith	3.8	1000	UCLA	Physics	Accepted
2	Jane Doe	3.9	800	Stanford	Biology	Rejected
3	Alice Brown	3.7	1200	Harvard	Economics	Accepted

Exercise-4

Exercise-4.1

Theta-Join

Theta Join

condition

$$Exp_1 \bowtie_\theta Exp_2 \equiv \sigma_\theta (Exp_1 \times Exp_2)$$

- **Explicit condition**: You specify exactly what to join on
- **Any condition allowed**: $=, <, >, \leq, \geq, \neq$
- **Any attributes**: Can join on attributes with different names

- Basic operation implemented in R-DBMS

  => Term "**join**" often means **theta-join**

Theta-Join

- The table corresponding to:

$$\sigma_{instructor.id = teaches.id}(instructor \times teaches))$$

$$=$$

$$instructor \bowtie_{Instructor.id = teaches.id} teaches.$$

Instructor.ID	*name*	*dept_name*	*salary*	*teaches.ID*	*course_id*	*sec_id*	*semester*	*year*
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
32343	El Said	History	60000	32343	HIS-351	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-101	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-319	1	Spring	2018
76766	Crick	Biology	72000	76766	BIO-101	1	Summer	2017
76766	Crick	Biology	72000	76766	BIO-301	1	Summer	2018
83821	Brandt	Comp. Sci.	92000	83821	CS-190	1	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-190	2	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-319	2	Spring	2018
98345	Kim	Elec. Eng.	80000	98345	EE-181	1	Spring	2017

Theta-Join

- The **join** operation allows us to combine a select operation and a Cartesian-Product operation into a single operation.
- Consider relations r (R) and s (S)
- Let "theta" be a predicate on attributes in the schema R X S.
- The join operation r $\bowtie_\theta$ s is defined as follows:

$$r \bowtie_\theta s = \sigma_\theta(r \times s)$$

- Thus

$$\sigma_{instructor.id = teaches.id}(instructor \times teaches))$$

- Can equivalently be written as

$$instructor \bowtie_{Instructor.id = teaches.id} teaches.$$

Theta-Join

Theta Join

$$Exp_1 \bowtie_\theta Exp_2 \equiv \sigma_\theta (Exp_1 \times Exp_2)$$

(condition)

- Theta-join: Condition can be <, <=, >, >=, <>, =
- We call Equi-Jon when the condition is =
- Basic operation implemented in R-DBMS
  => Term "**join**" often means **theta join**

Union Operation

- The union operation allows us to combine two relations
- Notation: $r \cup s$
- For $r \cup s$ to be valid.
  1. *r, s* must have the *same* **Degree** (same number of attributes)
  2. The attribute domains must be **Compatible**

- **Example**: Find all courses taught in the Fall 2017 semester, or in the Spring 2018 semester, or in both
- Result :

$$\Pi_{course_id}(\sigma_{semester=\text{“Fall”} \wedge year=2017}(section)) \cup \Pi_{course_id}(\sigma_{semester=\text{“Spring”} \wedge year=2018}(section))$$

course_id
CS-101
CS-315
CS-319
CS-347
FIN-201
HIS-351
MU-199
PHY-101

Exercise-5

The Rename Operation

- The results of relational-algebra expressions do not have a name that we can use to refer to them
- The rename operator, ρ, is provided for that purpose
- The expression:

  $\rho_{x(A1,A2, .. An)}(E)$

  returns the result of expression *E* under the name $\rho_{x(A1,A2, .. An)}$
- The expression:

  $\rho_{x}(E)$

  returns the result of expression *E* under the name *x*

The Rename Operation

Rename operator

1. $\rho_{R(A_1,\ldots,A_n)}(E)$
2. $\rho_{R}(E)$
3. $\rho_{A_1,\ldots,A_n}(E)$

1. **$\rho R(A_1,\ldots,A_n)(E)$**

Rename BOTH relation and attributes

2. $\rho R(E)$

Rename relation ONLY

3. $\rho A_1,\ldots,A_n(E)$

Rename attributes ONLY

Where:

E = expression/relation

R = new relation name

$A_1,\ldots,A_n$ = new attribute names

Relational Algebra: Exercise-5

List of college and student names

$$\rho_{C(name)}(\Pi_{cName}\ College) \cup \rho_{C(name)}(\Pi_{sName}\ Student)$$

```
C
name
---------
Stanford
Berkeley
MIT
Harvard
Alice
Bob
```

College

cName	state	enr

Student

sID	sName	GPA	HS

Apply

sID	cName	major	dec

Relational Algebra: Exercise-5.1

Students Table:

StudentID	Name	DeptID
1	Alice	10
2	Bob	20
3	Carol	10
4	David	20
5	Eve	10
6	Frank	30

Set-Intersection Operation

- The set-intersection operation allows us to find tuples that are in both the input relations.
- Notation: $r \cap s$
- Assume:
  - r, s have the *same degree*
  - attributes of r and s are compatible
- **Example**: Find the set of all courses taught in both the Fall 2017 and the Spring 2018 semesters.

$$\Pi_{course_id}\,(\sigma_{semester=\text{“Fall”} \wedge year=2017}\,(section)) \cap \Pi_{course_id}\,(\sigma_{semester=\text{“Spring”} \wedge year=2018}\,(section))$$

  - Result

course_id
CS-101

Set Difference Operation

- The set-difference operation allows us to find tuples that are in one relation but are not in another.
- Notation $r - s$
- => find tuples that are in r and those are not in s.
- Set differences must be taken between **compatible** relations.
  - r and s must have the same degree
  - attribute domains of r and s must be compatible
- **Example**: to find all courses taught in the Fall 2017 semester, but not in the Spring 2018 semester

$$\Pi_{course_id}(\sigma_{semester=\text{“Fall”} \wedge year=2017}(section)) - \Pi_{course_id}(\sigma_{semester=\text{“Spring”} \wedge year=2018}(section))$$

course_id
CS-347
PHY-101

Exercise-6

The Assignment Operation

- It is convenient at times to write a relational-algebra expression by assigning parts of it to temporary relation variables.
- The assignment operation is denoted by $\leftarrow$ and works like assignment in a programming language.
- **Example**: Find all instructor in the "Physics" and Music department.

$$Physics \leftarrow \sigma_{dept_name = \text{“Physics”}}(instructor)$$

$$Music \leftarrow \sigma_{dept_name = \text{“Music”}}(instructor)$$

$$Physics\ r \cup Music$$

- With the assignment operation, **a query can be written as a sequential program consisting of a series of assignments** followed by an expression whose value is displayed as the result of the query.

Equivalent Queries

- There is more than one way to write a query in relational algebra.
- Example: Find information about courses taught by instructors in the Physics department with salary greater than 90,000
- Query 1

$$\sigma_{dept_name = \text{“Physics”} \wedge salary > 90{,}000}(instructor)$$

- Query 2

$$\sigma_{dept_name = \text{“Physics”}}(\sigma_{salary > 90.000}(instructor))$$

- The two queries are not identical; they are, however, equivalent -- they give the same result.

Relational Algebra Summary

Core

R

$\sigma_C(E)$

$\pi_{A_1,\ldots,A_n}(E)$

$(E_1) \times (E_2)$

$E_1 \cup E_2$

$E_1 - E_2$

$\rho_{R(A_1,\ldots,A_n)}(E)$

$E_1 \bowtie E_2$

$E_1 \bowtie_\theta E_2$

$E_1 \cap E_2$